

Code as Design

Juraj Sulimanović

Software
Engineering



Software
Design

“Engineering is the creative application of science to solve problems”



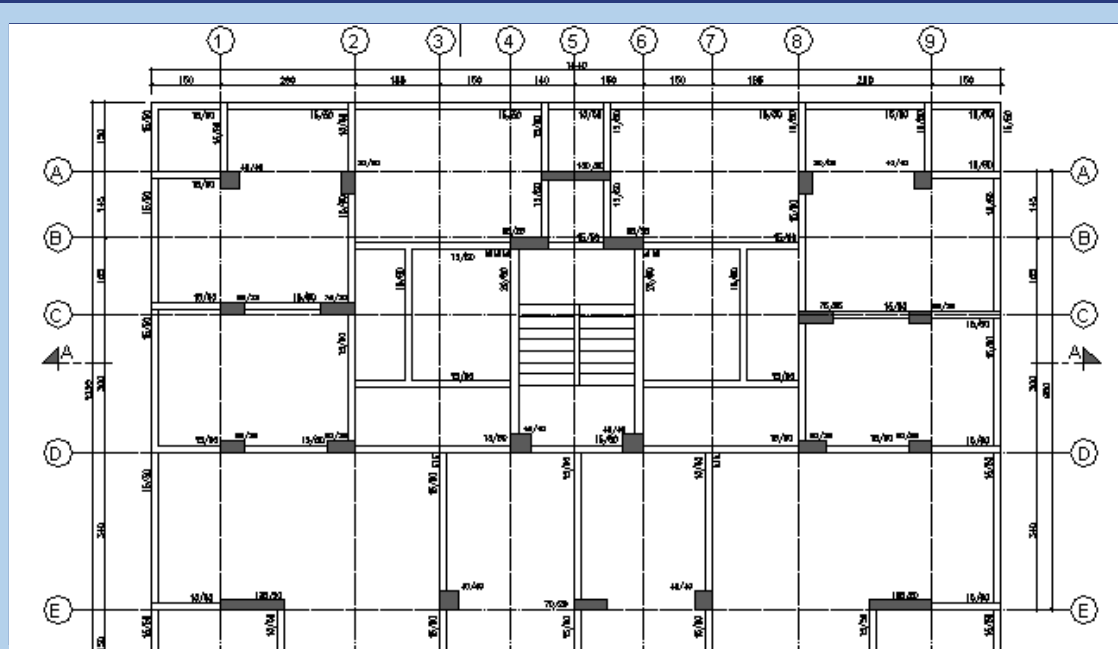
“Design is the creation of a plan or specification for the construction of an object or a system.”

Engineering
activity

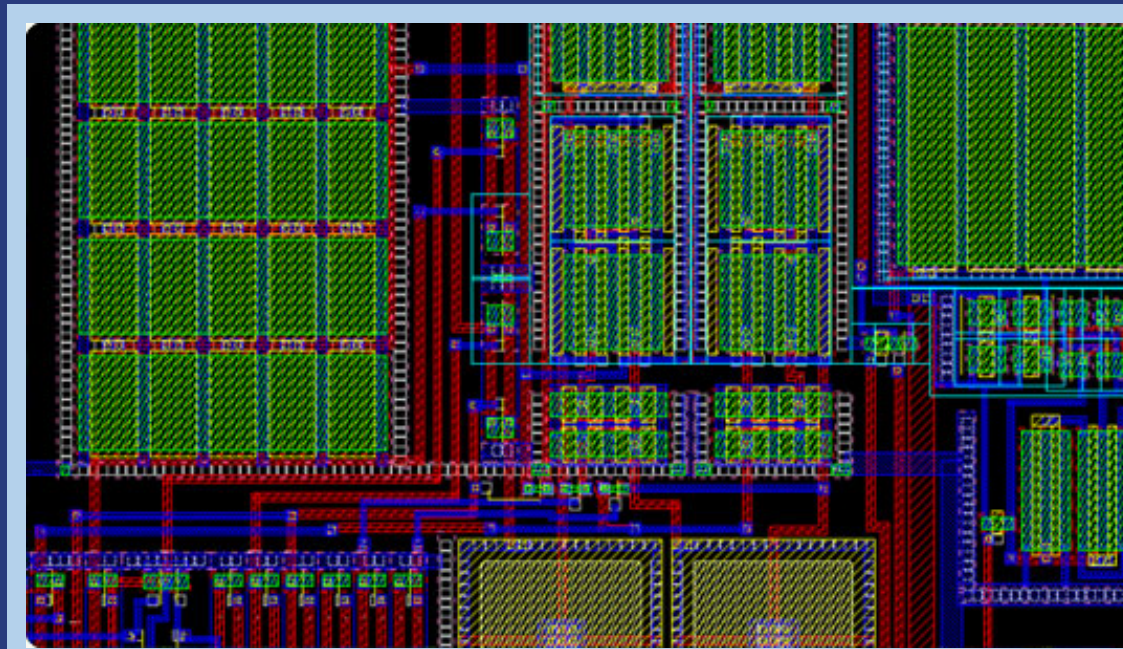


Design
document

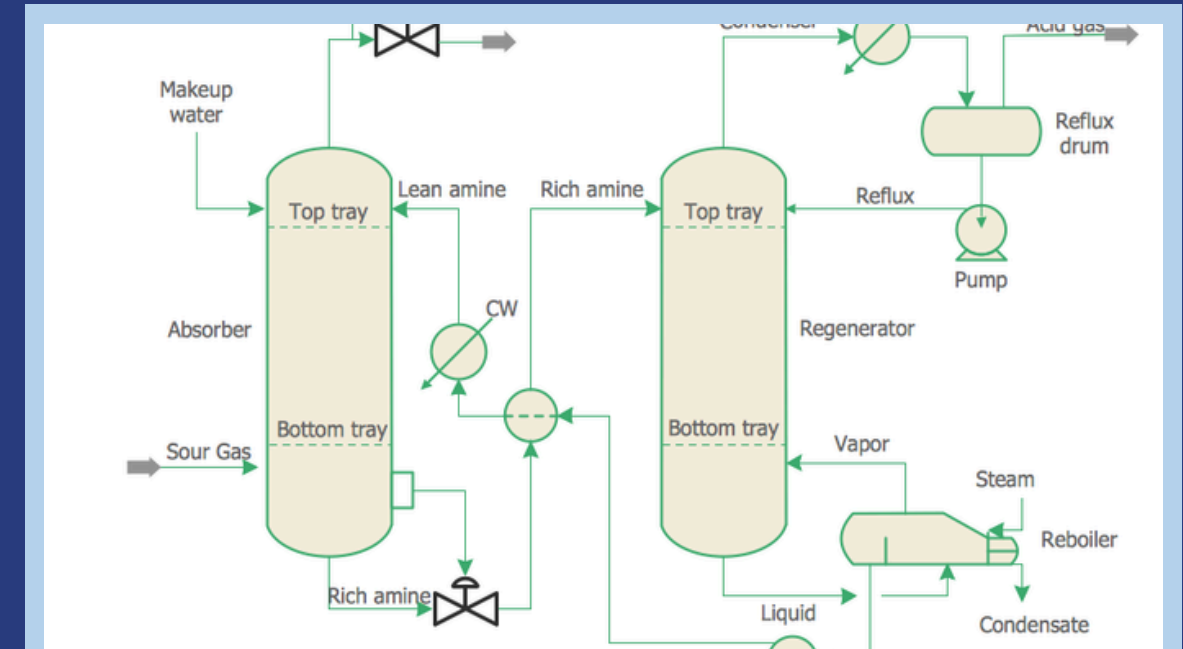
Civil Engineering



Electronic Engineering



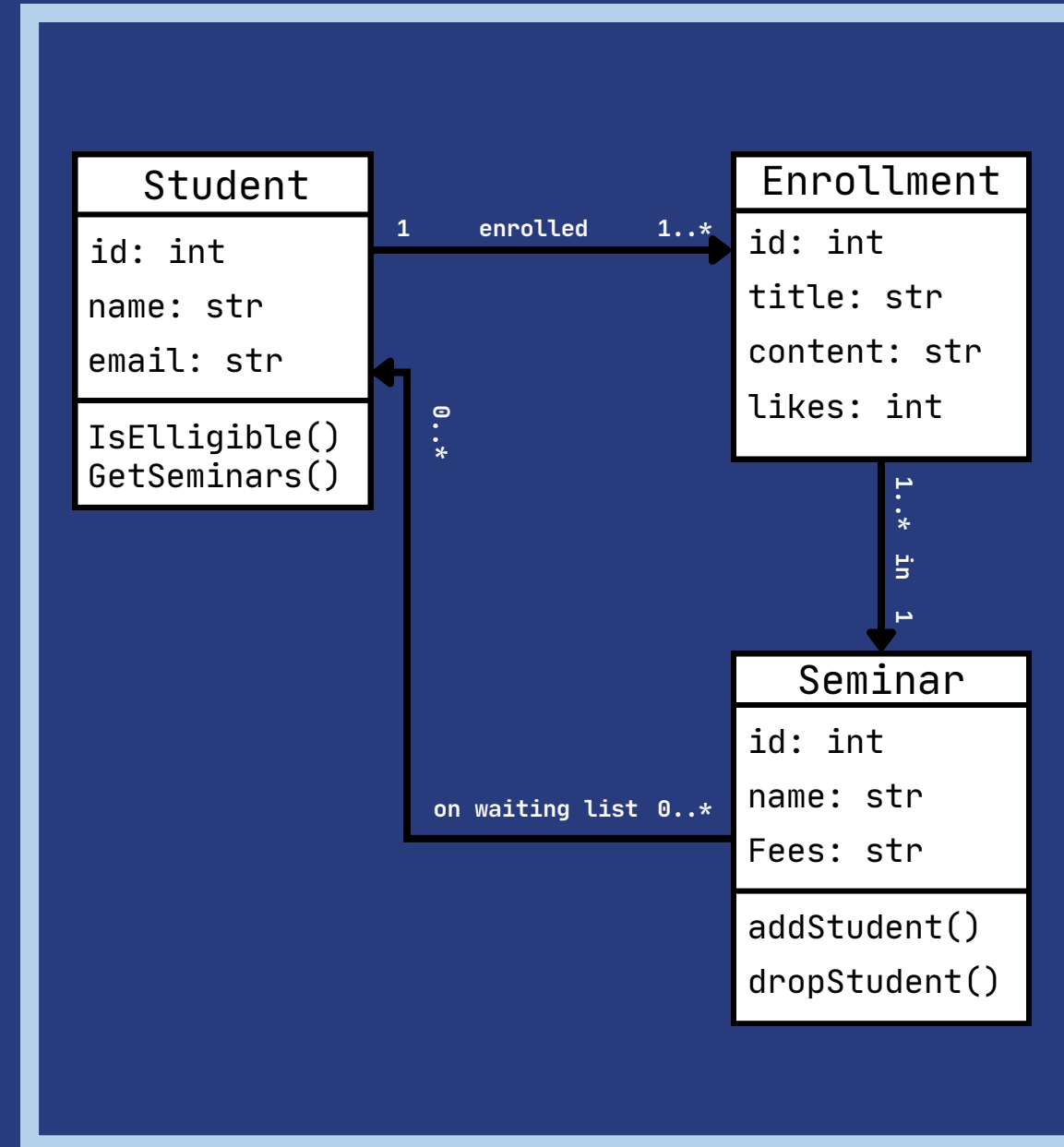
Chemical Engineering



Software Engineering

?

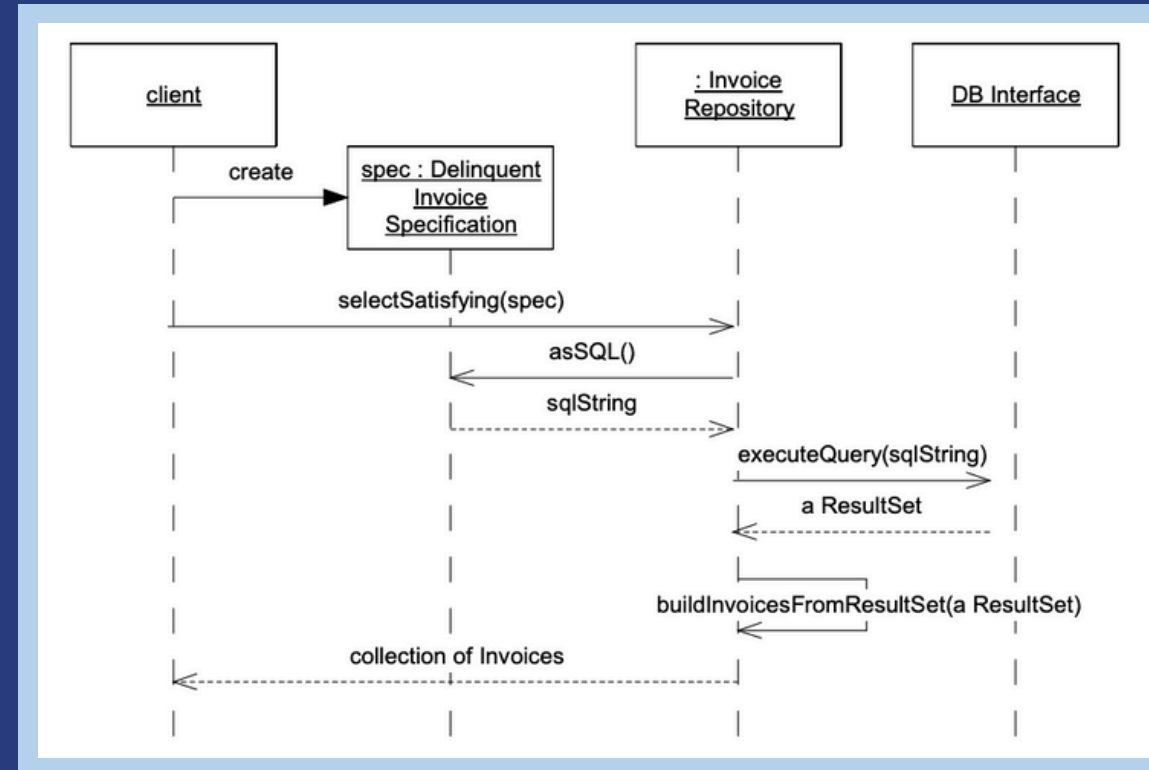
Software Engineering



Software Engineering

Technical Design Document IDA-MS-TD Issue 1	Page 5
1	INTRODUCTION
#1	<i>This section should provide an overview of the entire document and a description of the scope of the system and its intended usage. The scope should also describe external interfaces to the system, external dependencies and provide a brief overview of the 'characteristics' of the system, commenting on aspects such as real-time use, security considerations, concurrency of users etc.</i>
#2	<i>The operating system, development language to be used for the system development and any COTS packages that will be used, such as databases etc. should also be referred to in the introduction. This should be sufficient for the casual reader to gain a good appreciation of the key building blocks of the system. The reader should also be introduced to the security measures that need to be included within the system. Where strong authentication models are required this may need to show how aspects such as authentication of users may need to be implemented using PKI for example.</i>
1.1	PURPOSE
#1	<i>This section should:</i> <i>a. describe the purpose of this document;</i> <i>b. specify the intended readership of this document.</i>
1.2	SCOPE
#1	<i>This section should:</i> <i>a. identify the products to be produced;</i> <i>b. explain what the proposed system will do (and will not do, if necessary);</i> <i>c. define relevant benefits, objectives and goals as precisely as possible;</i> <i>d. define any security risks associated with the system;</i> <i>e. be consistent with similar statements in higher-level specifications, if they exist.</i>
1.3	DEFINITIONS, ACRONYMS AND ABBREVIATIONS
#1	<i>This section should define all terms, acronyms and abbreviations used in this document. Particular care should be taken to define terms that are specific to the application.</i>
#2	<i>Terms covering the development of software using the Unified Modelling Language (UML) approach – which is recommended if an OOD view of the system is to be developed – should be included for completeness.</i>

Software Engineering



- In reality, these become obsolete when implementation begins.
- In engineering, when a design is complete, the design documentation is turned over to the manufacturing team.

The developers?

- Building software is done by compilers, linkers and interpreters.
- They build it according to the design, which is the source code

Ruby Bytecode

Ruby Source Code

```
class Person
  attr_accessor :name, :age

  def initialize(name, age)
    @name = name
    @age = age
  end
end
```



```
== disasm: #<ISeq:<compiled>@example.rb:1 (1,0)-(8,3)>
0000 putspecialobject 3 ( 1)[Li]
0002 putnil
0003 defineclass :Person, <class:Person>, 0
0007 leave
```

```
== disasm: #<ISeq:<class:Person>@example.rb:1 (1,0)-(8,3)>
0000 putself ( 2)[LiCl]
0001 putobject :name
0003 putobject :age
0005 opt_send_without_block <calldata!mid:attr_accessor,
argc:2, FCALL|ARGS_SIMPLE>
0007 pop
0008 definemethod :initialize, initialize ( 4)[Li]
0011 putobject :initialize
0013 leave ( 8)[En]
```

```
== disasm: #<ISeq:initialize@example.rb:4 (4,2)-(7,5)>
local table (size: 2, argc: 2 [opts: 0, rest: -1, post: 0,
block: -1, kw: -1@-1, kwrest: -1])
[ 2] name@0<Arg>[ 1] age@1<Arg>
0000 getlocal_WC_0 name@0 ( 5)[LiCa]
0002 setinstancevariable :@name, <is:0>
0005 getlocal_WC_0 age@1 ( 6)[Li]
0007 dup
0008 setinstancevariable :@age, <is:1>
0011 leave ( 7)[Re]
```



```
== disasm: #<ISeq:<compiled>@example.rb:1 (1,0)-(8,3)>
0000 putspecialobject 3 ( 1)[Li]
0002 putnil
0003 defineclass :Person, <class:Person>, 0
0007 leave
```

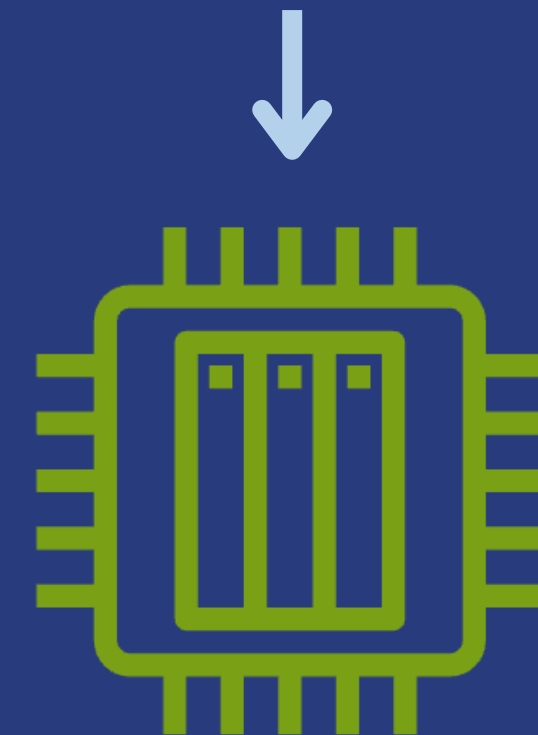
```
== disasm: #<ISeq:<class:Person>@example.rb:1 (1,0)-(8,3)>
0000 putself ( 2)[LiCl]
0001 putobject :name
0003 putobject :age
0005 opt_send_without_block <calldata!mid:attr_accessor,
argc:2, FCALL|ARGS_SIMPLE>
0007 pop
0008 definemethod :initialize, initialize ( 4)[Li]
0011 putobject :initialize
0013 leave ( 8)[En]
```

```
== disasm: #<ISeq:initialize@example.rb:4 (4,2)-(7,5)>
local table (size: 2, argc: 2 [opts: 0, rest: -1, post: 0,
block: -1, kw: -1@-1, kwrest: -1])
[ 2] name@0<Arg>[ 1] age@1<Arg>
0000 getlocal_WC_0 name@0 ( 5)[LiCa]
0002 setinstancevariable :@name, <is:0>
0005 getlocal_WC_0 age@1 ( 6)[Li]
0007 dup
0008 setinstancevariable :@age, <is:1>
0011 leave ( 7)[Re]
```



x86 instruction set

```
section .data
    msg db call, 0xA
    len equ $ - msg
section .text
    global _start
_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, msg
    mov edx, len
    int 0x80
    mov eax, 1
    xor ebx, ebx
    int 0x80
```



Building the design is essentially free.

Building the design is essentially free.

- software is very dynamic with constantly changing requirements

Building the design is essentially free.

- software is very dynamic with constantly changing requirements
- the source code is the only design which is in sync with the product

Building the design is essentially free.

- software is very dynamic with constantly changing requirements
- the source code is the only design which is in sync with the product
- designs tend to be incredibly large and complex

How do we deal with complexity, what makes a good design?

What makes a good design?



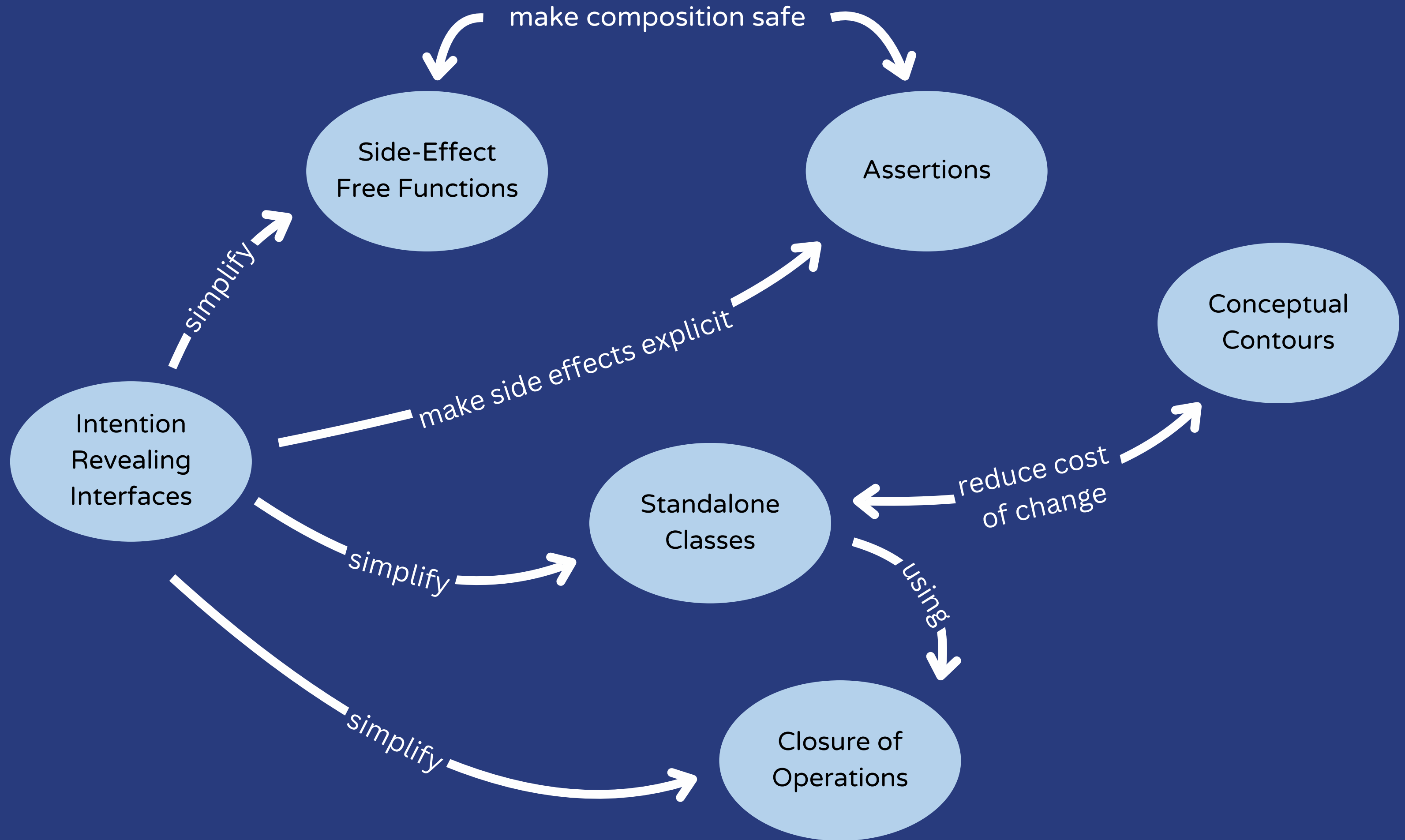
What makes a good design?

a supple design which closely models the real-world
domain of the problem you're trying to solve

What makes a good design?

It's not design patterns or "clean code"

it's model driven design




```
module PaintMixing
  class Paint
    attr_accessor :volume, :red, :yellow, :blue

    def initialize(volume, red, yellow, blue)
      @volume = volume
      @red = red
      @yellow = yellow
      @blue = blue
    end

    def mix_in(other)
      total_volume = volume + other.volume
      ratio = other.volume / total_volume

      @red      = (1 - ratio) * @red      + ratio * other.red
      @yellow   = (1 - ratio) * @yellow   + ratio * other.yellow
      @blue     = (1 - ratio) * @blue     + ratio * other.blue
      @volume = total_volume
      self
    end
  end
end
```

```
module PaintMixing
  class PigmentColor
    include ActiveModel::Validations
    attr_reader :red, :yellow, :blue

    validates :red, :yellow, :blue,
              presence: true,
              numericality: { only_integer: true, greater_than_or_equal_to: 0, less_than_or_equal_to: 255 }

    def initialize(red, yellow, blue)
      @red = red
      @yellow = yellow
      @blue = blue
    end

    def mixed_with(other, ratio)
      return self if other.nil?

      new_red      = ((1 - ratio) * red      + ratio * other.red).round
      new_yellow   = ((1 - ratio) * yellow   + ratio * other.yellow).round
      new_blue     = ((1 - ratio) * blue     + ratio * other.blue).round

      PigmentColor.new(new_red, new_yellow, new_blue)
    end
  end
end
```

```
module PaintMixing
  class Paint
    include ActiveModel::Validations
    attr_reader :volume, :pigment_color

    validates :volume, numericality: { greater_than: 0 }
    validates :pigment_color, presence: true

    def initialize(volume, pigment_color)
      @volume = volume
      @pigment_color = pigment_color
    end

    def mix_in(other)
      new_volume = volume + other.volume
      ratio = other.volume.to_f / new_volume
      new_color = pigment_color.mixed_with(other.pigment_color, ratio)

      Paint.new(new_volume, new_color)
    end
  end
end
```

To create a supple design we have to:

- Shake off excessive up-front design
- Gain a deep understanding of the domain
- Refactor and do iterative development

Traditional documentation has its use

- Auxiliary
- Temporary
- Used for communication

Thank you